Zhao et al. *EPJ Data Science* (2025) 14:1
https://doi.org/10.1140/epjds/s13688-024-00517-7

EPJ Data Science
a SpringerOpen Journal

EPJ.org

RESEARCH Open Access

A hybrid stock prediction method based on periodic/non-periodic features analyses

Cheng Zhao[1*], Junyi Cai[2] and Shuyi Yang[2]

*Correspondence:
zhaoc@zjut.edu.cn
[1]School of Economics, Zhejiang University of Technology, Hangzhou, China
Full list of author information is available at the end of the article

Abstract

Stock investment is an economic activity characterized by high risks and high returns. Therefore, the prediction of stock prices or fluctuations is of great importance to investors. Stock price prediction is a challenging task due to the nonlinearity and high volatility of stock time series. Existing deep learning models may not capture the periodic and non-periodic features of stock data effectively. In this paper, we propose a novel model that leverages Complete Ensemble Empirical Mode Decomposition (CEEMD), Time2Vec, and Transformer to better capture and utilize various patterns in stock data for enhanced prediction performance, and we call it ETT. CEEMD decomposes the stock data into different frequency components based on their intrinsic scales. Time2Vec provides a time vector representation that captures both periodic and non-periodic patterns while being invariant to time scaling. Transformer learns the long-term dependencies and global information from the data. We apply ETT to predict stock prices in the Chinese A-share market and compare it with several baseline models. The results show that ETT reduces the mean squared error (MSE) by an average of 4% and increases the average cumulative return by 58% on the CSI 100 and Hushen 300 datasets.

Keywords: CEEMD; Time2Vec; Transformer; Periodic/Non-Periodic Features

1 Introduction

The stock market is an important component of a market economy and serves as an economic barometer. With the development of the economy and the improvement of people's investment awareness, an increasing number of individuals are participating in it. However, the fluctuations in stock prices are influenced by various factors such as market conditions, politics, macroeconomics, and investor psychology [1, 2]. These factors result in a mixture of periodic and non-periodic features, characterized by nonlinearity and instability [3]. Additionally, the data volume is substantial, and there is a high level of noise [4]. For investors, the ability to make timely and accurate judgments on stock price changes in advance enables better preparation. Therefore, accurately predicting stock prices is a highly challenging yet crucial task.

Early stock price prediction models employed traditional econometric methods, including Generalized Autoregressive Conditional Heteroskedasticity (GARCH) and Auto-

Springer

Regressive Moving Average Model (ARMA) models. Although these traditional econometric models achieved some success in the field of stock prediction [5, 6], subsequent research has shown that these models struggle to capture the non-periodic features of stocks and fail to fully reflect the true distribution of the data [7]. Machine learning models enable computers to learn patterns from data and make predictions on future data based on these patterns. Common machine learning models for stock prediction include Random Forest, Support Vector Machines (SVM), and Gradient Boosting Decision Trees (GBDT) [8]. However, in scenarios with large amounts of data, machine learning models often exhibit poor prediction performance [9]. Deep learning, as a further development of machine learning, encompasses popular models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory Networks (LSTM). Vaswani et al. proposed an innovative deep learning architecture called Transformer [10], which utilizes attention mechanisms to process data and enables parallel training while capturing global information more effectively. However, when using the Transformer to handle time series data, the entire time series is forwarded all at once through the Transformer architecture, making it difficult to extract temporal or sequential dependencies and features [11]. To overcome temporal differences in the Transformer, Kazemi et al. [12] introduced a model-agnostic time vector representation called Time2Vec. Time2Vec captures both periodic and non-periodic features, generating time vector representations. This embedding layer can be incorporated into the Transformer to enhance model performance.

Research has shown that frequency decomposition algorithms can assist deep learning models in better analyzing the decomposed time series [13]. One common approach is to apply one-shot decomposition methods widely used in time series modeling to improve prediction performance. However, the stability of the subseries obtained after one-shot decomposition may be compromised [14], and they could even contain spurious signals. In such cases, directly modeling the subseries may not lead to improved accuracy. Models that combine traditional decomposition algorithms with deep learning models offer greater advantages [15, 16]. Huang, Shen, Long, et al. [17] proposed Empirical Mode Decomposition (EMD), an effective method for handling non-linear and non-stationary time series. Any time series can be decomposed into a finite number of Intrinsic Mode Functions (IMFs) through EMD, where each IMF represents a component of the original signal at different feature scales. These components form a complete and nearly orthogonal basis for the original signal. Complete empirical ensemble mode decomposition (CEEMD), an improvement over EMD, employs paired positive and negative auxiliary white noise to enhance the decomposition process. Both EMD and CEEMD are adaptive and efficient. Thus, frequency decomposition methods such as EMD and CEEMD serve as useful tools to reduce the complexity of data sequences. They can separate highly volatile data into different smaller frequency components [18] and play a useful role in financial time series analysis and deep learning models [19]. For instance, Chi, Guan, et al. [20] proposed a hybrid model that combines Empirical Mode Decomposition (EMD) and attention-based Long Short-Term Memory (LSTM) to improve financial time series prediction accuracy. Similarly, Yao, Zhang, and Zhao [21] introduced the MEMD-TCN model, which applies Multivariate Empirical Mode Decomposition (MEMD) to break down stock index data into multiple frequency subsequences, with Temporal Convolutional Networks (TCN) then employed for time series forecasting.

Inspired by this, this paper proposes a novel stock prediction model, CEEMD-Time2-Vec-Transformer (ETT). This model combines the methods that can capture both periodic and non-periodic features in stock time series, namely CEEMD and Time2Vec, and utilizes the Transformer for prediction.

The remainder of this paper is organized as follows. Section 2 provides a review of related work. Section 3 explains the proposed methodology. Section 4 analyzes the experimental results. The final section concludes with a summary and future prospects.

Our contributions can be summarized as follows:

(1) A new CEEMD-Time2Vec-Transformer (ETT) model for stock market prediction is proposed, effectively capturing both periodic and non-periodic characteristics in stocks to enhance the predictive performance of the model.

(2) Experiments are conducted on the CSI 100 and Hushen 300 datasets, validating the effectiveness of the model and performing backtesting. The comparison results against other models demonstrate that our model can achieve higher excess returns.

2 Related work

This chapter reviews the relevant literature, including the development of deep sequential learning algorithms in stock prediction, the contributions of frequency decomposition algorithms in addressing time series-related problems, and their integration with deep learning models.

According to [22], the primary information in the stock market is reflected in price changes. Therefore, modeling and predicting stock prices directly can be viable. Early researchers constructed stock prediction models based on traditional machine learning algorithms. For example, Kaczmarek et al. [23] applied the Random Forest (RF) method to predict the constituents of the S&P 500, validate forecast volatility, and analyze the efficiency of portfolio optimization. Yu et al. [24] introduced Principal Component Analysis (PCA) into the SVM model to extract low-dimensional feature information. However, these machine learning model structures may struggle to capture the various feature relationships between sequence heterogeneity [11].

With the development of deep learning, many existing research results have demonstrated that deep learning frameworks outperform traditional machine learning models in financial market prediction. Typical network architectures, such as CNN and RNN, have better predictive capabilities than traditional machine learning algorithms. In paper [25], the LSTM model and Bidirectional LSTM (BiLSTM) model were used with appropriate hyperparameter tuning to predict future stock trends. Deep learning frameworks can learn patterns of temporal variation from historical data, enabling better prediction of future price changes. Therefore, it is feasible to utilize deep learning for forecasting stock and forex market trends [26]. By utilizing an autoencoder composed of stacked restricted Boltzmann machines, Takeuchi et al. [27] achieved an accuracy of 53.36% in predicting the direction of US stocks. Sirignano et al. [28] predicted the National Association of Securities Dealers Automated Quotations (NASDAQ) stock index using a Multilayer Perceptron (MLP) and a hybrid artificial neural network. Kaushik and Giri [29] compared LSTM with traditional econometrics, vector auto-regression, and support vector machine to predict exchange rate fluctuations. They found that LSTM outperformed SVM and VAR methods in analysis and prediction. Selvin et al. [30] used three different deep learning architectures (LSTM, RNN, and CNN) for price prediction of companies listed on the Na-

tional Stock Exchange (NSE) of India Limited. Vaswani et al. proposed the groundbreaking Transformer architecture [3], which achieved significant success in Natural Language Processing (NLP) problems. After that, Li et al. [31] enhanced the locality and broke the memory bottleneck of Transformer on time series forecasting. Zhou et al. [32] introduced an improved Transformer model for long sequence time series prediction with significant effects. Muhammad et al. [33] achieved good results by using Transformer to predict the future price of stocks on the Dhaka Stock Exchange (DSE), the leading stock exchange in Bangladesh. Zhou et al. [34] used Transformer to predict the stock market index. Ding et al. [35] proposed a Transformer-based method for the stock movement prediction task. Malibari et al. [36] introduced a caple network based on Transformer Encoder model to extract deep semantic features from social media and then capture the structural relationships of the text using capsule network to predict stock prices. Based on the above research, it can be concluded that Transformer models are one of the accepted deep learning models in the financial area. The aforementioned work demonstrates the successful application of traditional deep learning models in the financial domain. However, traditional neural networks may not adequately learn various effective data feature representations [37], thus reducing the accuracy of analysis.

In paper [14], it was pointed out that combining Transformer with traditional signal analysis algorithms has significant advantages. This is because deep learning models cannot consistently and stably produce high-quality predictions in all scenarios. Utilizing signal analysis methods for time decomposition and feature extraction can enhance the effectiveness of the model. Typical signal decomposition algorithms such as EMD and CEEMD are quite effective in handling nonlinear and non-stationary financial time series [38]. They can clearly decompose large fluctuating time series into smaller frequency components, thereby addressing the non-stationarity and nonlinearity of the input data [30]. Wang et al. [39] combines EMD and Stochastic Time Strength Neural Network (STNN) to forecast stock price fluctuations, yielding better performance than benchmark models. Furthermore, Time2Vec [12] is an orthogonal yet complementary approach that provides a model-agnostic vector representation for time. Time2Vec can capture both periodic and non-periodic patterns while maintaining stability in the face of temporal variations. The introduction of Time2Vec offers a time vector representation for time series, thereby improving the effectiveness of using Transformer for stock sequence prediction. For instance, Dang et al. [40] combines Time2Vec, Gate Recurrent Unit (GRU), and Fully Connected Neural Network (FCN) for stock price prediction, achieving 56.4% accuracy. Muhammad et al. [33] successfully applies a Transformer model combined with Time2Vec to predict the price of stocks in the DSE and obtains favorable results.

Inspired by these findings, this paper proposes a novel Transformer-based stock prediction model, ETT. This model combines CEEMD, which enables signal decomposition based on the intrinsic time scale features of the data, and Time2Vec, which captures the non-periodic patterns of the data while remaining invariant to time scaling. This combination allows for more effective extraction of pattern information from stock time series, thereby enhancing the accuracy of predictions.

3 Methodology

In this work, we focus on forecasting stock fluctuations. Formally, we use the historical observation sequence of stock data $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\} \in \mathbb{R}^{T \times F}$, where T is the time steps

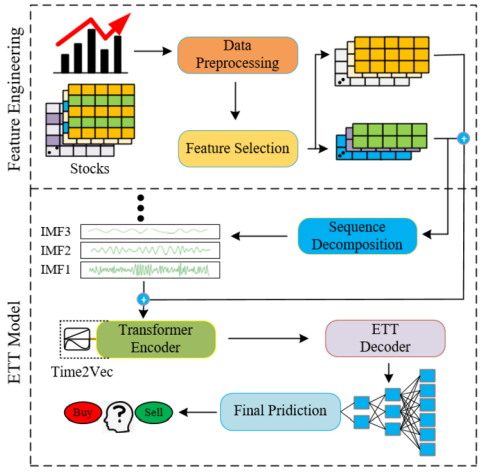

Figure 1 Overall architecture diagram of ETT: Stock data, processed via feature engineering, is decomposed by CEEMD. The decomposed signals, combined through residual connections, are input into a Transformer model, producing final predictions via the ETT Decoder

and F is the number of features. For convenience, we denote $\mathbf{X}_{t,:}$ as the values recorded at time step t, and $\mathbf{X}_{:,f}$ as the entire time series of feature f. The model takes the feature matrix $\mathbf{X}$ as input and outputs the predicted label $\mathbf{y}$. The calculation of $\mathbf{y}$ is shown as follows, where p_T represents the closing price at time T.

$$\mathbf{y} = \frac{p_T - p_{T-1}}{p_T} \tag{1}$$

3.1 Architecture of ETT

After feature engineering, the stock data is input into the ETT model. First, the sliding window method is used to collect the required data, and CEEMD is applied for frequency decomposition on this data. The decomposed signals are then combined with the original signal through residual connections. Subsequently, the decomposed data is fed into the Transformer model. During the backpropagation process, Time2Vec automatically captures non-periodic patterns in the stock time series, and the final prediction results are output through the Decoder layer. The overall process is illustrated in Fig. 1.

3.2 Feature engineering

Since outliers can affect the estimation of correlation between features and label, it is necessary to handle these outliers. We used the $n\sigma$ method to perform outlier removal on the data. First, we calculated the mean μ_f and standard deviation σ_f for each feature. Then, we determined the threshold parameter n, commonly set to 3. Finally, we adjusted the feature values that exceeded the range $[\mu_f - 3\sigma_f, \mu_f + 3\sigma_f]$ by setting the values exceeding the threshold to the threshold value. The values within the threshold range remained unchanged.

To eliminate the influence of dimensional units among features, the data needs to be standardized. After applying data standardization, the distribution of the original data becomes a standard normal distribution, which is suitable for comprehensive comparative

evaluation. The formula for zero-mean normalization is as follows:

$$\mathbf{X}'_{:,f} = \frac{\mathbf{X}_{:,f} - \mu_f}{\sigma_f} \tag{2}$$

where $\mathbf{X}'_{:,f}$ is normalization value of the f-th feature, and we use the parameters obtained from the training set to standardize the validation set as well.

3.3 ETT model

3.3.1 Complete Empirical Ensemble Mode Decomposition (*CEEMD*)

Torres et al. [41] extended the previous frequency decomposition model and proposed CEEMD. CEEMD suppresses the mode mixing problem in traditional EMD decomposition by introducing pairs of positive and negative white noise. In this section, CEEMD is used to decompose the time series of each feature in the feature matrix. CEEMD not only alleviates mode mixing but also reduces the reconstruction error of stock label sequences caused by white noise. The specific algorithmic process is as follows:

(1) Initialize parameters: Initialize the number of ensemble iterations I and the amplitude of white noise for CEEMD. For each feature $\mathbf{X}'_{:,f}$, we will randomly generate pairs of positive and negative white noise n_i, where $i = \{1, 2, \ldots, I\}$ represents the number of iterations.

(2) Add white noise: In each iteration, white noise is added to produce two signals, as shown below:

$$\mathbf{P}_i = \mathbf{X}'_{:,f} + n_i \qquad \mathbf{N}_i = \mathbf{X}'_{:,f} - n_i \tag{3}$$

(3) EMD decomposition: Perform EMD decomposition on the signals $\mathbf{P}_i$ and $\mathbf{N}_i$, which include added positive and negative white noise, respectively. This results in q intrinsic mode functions (IMFs), which can be represented as follows:

$$\mathbf{P}_i = \sum_{j=1}^{q} c_{j,i}^{(1)} \qquad \mathbf{N}_i = \sum_{j=1}^{q} c_{j,i}^{(2)} \tag{4}$$

The variables $c_{j,i}^{(1)}$ and $c_{j,i}^{(2)}$ represent the j-th IMF component obtained after the i-th EMD decomposition.

(4) Calculate the mean of the IMFs: After completing each iteration, compute the average of all IMF components to reduce the impact of noise. The mean of the j-th IMF component can be represented as follows:

$$\mathbf{c}_j = \frac{1}{2I} \sum_{i=1}^{I} c_{j,i}^{(1)} + c_{j,i}^{(2)} \tag{5}$$

The variable $\mathbf{c}_j$ represents the j-th IMF component obtained from CEEMD decomposition.

To address potential issues with future data, a sliding window approach is employed to obtain the IMF values for daily data. For each iteration, all features from the previous T days of the stock sequence are decomposed using CEEMD. The decomposed sequence is then combined with the original sequence via residual connections. Figure 2 illustrates the

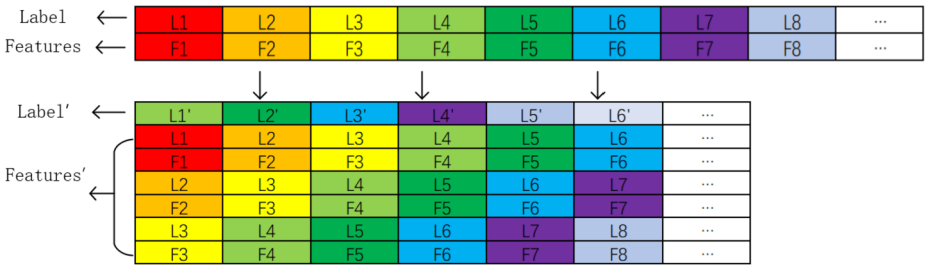

Figure 2 Features and labels are constructed from the stock sequence using the sliding window approach. Different colors represent the stock sequence information for different dates

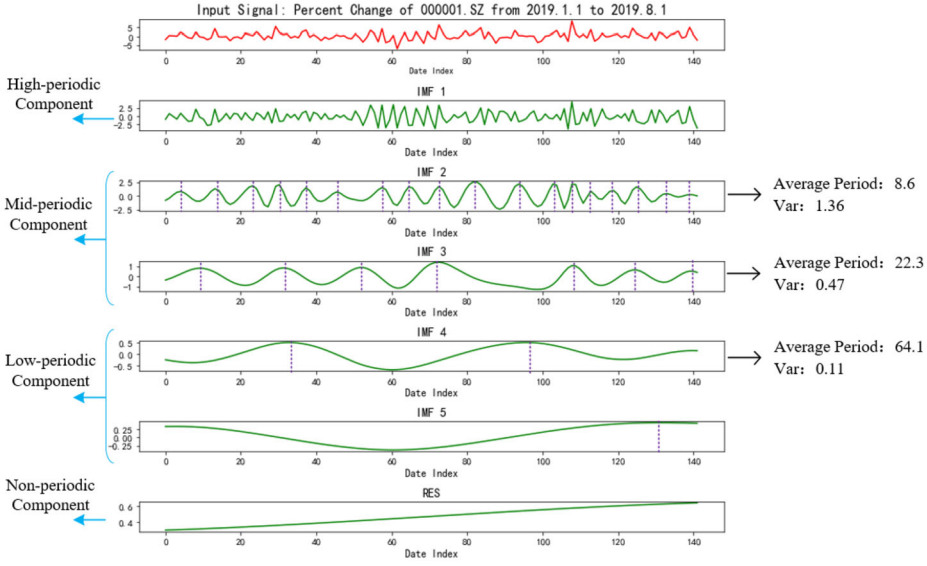

Figure 3 The IMF components of stock 000001.SZ from 2019.1.1 to 2019.8.1. Multiple Intrinsic Mode Functions (IMFs) are presented, corresponding to high, mid, and low-frequency components, as well as a non-periodic residual. The average period and variance displayed on the right elucidate the inherent frequency characteristics of the stock data

process of constructing features and labels from a single stock, using $T = 3$ as an example, where $\mathbf{F}_t$ represents all features on day t, and $\mathbf{L}_t$ represents the label on day t. During the construction process, the first new data point is formed by combining all features and labels from the first three days of the original dataset into a new set of features. These new features are then decomposed using CEEMD, with the label for the fourth day serving as the new label. The subsequent new data points are generated in a similar manner.

Figure 3 demonstrates the CEEMD decomposition and IMFs information obtained from stock 000001.SZ from January 1, 2019, to August 1, 2019. The CEEMD algorithm decomposes complex stock time series into a finite number of intrinsic mode functions. Each IMF component contains local feature information at different time scales from the original sequence and exhibits a higher signal-to-noise ratio. The IMFs can be categorized into high-frequency, mid-frequency, low-frequency, and non-periodic components based on their frequencies. Each component exhibits a relatively stable cycle. For example, IMF2 in

Fig. 3 has an average period of 8.6 and a variance of 1.36, while IMF4 has an average period of only 22.3 and a variance of 0.47. Such decomposition is advantageous for data analysis.

Finally, we obtain the IMFs corresponding to each feature. Next, these IMFs are combined with the original sequence via residual connections to generate new feature representations. After this process, the feature dimension is expanded from F to F'. The entire process can be expressed as follows:

$$\hat{\mathbf{X}} = \text{Res}(\text{CEEMD}(\mathbf{X}')) \tag{6}$$

3.3.2 *Embeddings and positional encoding*

Embedding is typically used in natural language processing to map sparse or high-dimensional word vectors to a lower-dimensional space [42]. Similarly, we convert the processed data into embedding representations through the embedding layer. Given the input data $\hat{\mathbf{X}}$, the feature sequence generates embedding vectors after passing through the embedding layer. To effectively represent the inherent temporal information in stock time series data, we add positional encoding to the input sequence. Specifically, we use sine and cosine functions of different frequencies to compute these positional encodings, as shown below:

$$\mathbf{PE}_{(t,2i)} = \sin(t/10{,}000^{2i/d_{model}}) \tag{7}$$

$$\mathbf{PE}_{(t,2i+1)} = \cos(t/10{,}000^{2i/d_{model}}) \tag{8}$$

Where d_{model} is the dimension of each embedding vector, t is the position of each vector, and i denotes the index of the frequency. After embedding and positional encoding, the final output $\mathbf{H}$ can be represented as follows:

$$\mathbf{H} = \text{Embedding}(\hat{\mathbf{X}}) + \mathbf{PE} \tag{9}$$

3.3.3 *Time2Vec*

Since stock sequences can be measured across different time scales (such as daily, weekly, monthly, etc.), an important property of time representations is invariance to time scaling [43]. Time2Vec excels in handling multi-scale stock sequence data by maintaining stability under time scaling, making it particularly suitable for use in conjunction with Transformers. When processing time series data, Time2Vec can effectively capture both periodic and non-periodic features in stock sequences, thereby enhancing the model's adaptability to different time patterns.

Time2Vec is an orthogonal but complementary approach, which provides a model-agnostic vector representation for time.

$$t2v\,(\mathbf{H})\,[i] = \begin{cases} \omega_i \mathbf{H} + \varphi_i, & i = 0 \\ F\,(\omega_i \mathbf{H} + \varphi_i\,), & 1 \le i \le k \end{cases} \tag{10}$$

Where k represents the dimension of Time2Vec, F is the periodic activation function, ω_i and φ_i are a set of learnable parameters that serve as the weights and biases in the embedding layer of Time2Vec. Therefore, the results of Time2Vec embedding can capture both periodic and non-periodic patterns. For the periodic components, the function

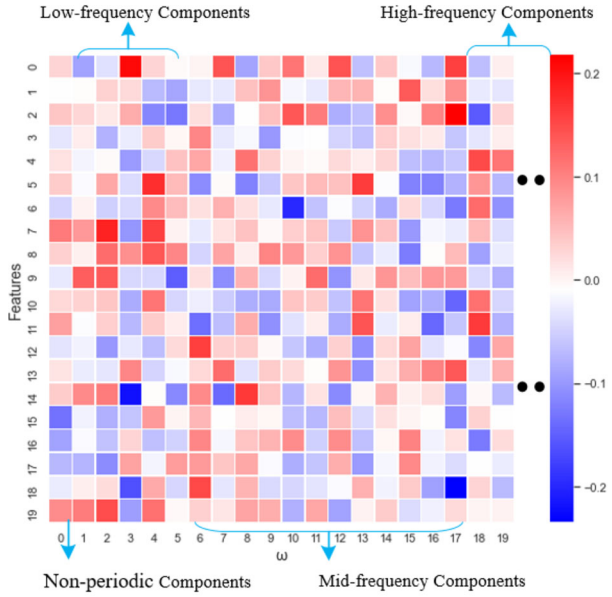

Figure 4 The heatmap of a partial result of the parameter ω_i in Time2Vec for stock 000001.SZ after training the ETT model

$F(\omega_i \mathbf{H} + \varphi_i)$ has a period of $2\pi/\omega_i$, which means that for $\mathbf{H}$ and $\mathbf{H} + 2\pi/\omega_i$, the output values of the function are the same. Sine or cosine functions help capture multi-scale periodic behaviors in stock sequences, especially in the different frequency IMF sequences obtained after CEEMD decomposition. For example, in the sine function $sin(\omega_i \mathbf{H} + \varphi_i)$, when $\omega_i = 2\pi/20$, it indicates that every 20 sequences repeat, making it suitable for capturing monthly patterns. Furthermore, the sine function can effectively extrapolate future data. The linear component (corresponding to $i = 0$) represents the non-periodic changes in time and is used to capture the non-periodic patterns in the input data. After processing with Time2Vec, the feature sequence generates a new temporal embedding representation, expressed as follows:

$$\mathbf{Z} = \mathbf{H} + \mathrm{t2v}(\mathbf{H}) \tag{11}$$

Taking stock 000001.SZ as an example, Fig. 4 displays a heatmap created by plotting a subset of the parameters ω_i learned by Time2Vec. From the figure, it can be observed that the model captures both periodic and non-periodic data. The brightness at $\omega = 0$ is moderate, indicating a lower presence of non-periodic components in the data. The low-frequency portion exhibits brighter colors, suggesting a higher abundance of low-frequency data. In the mid-to-high-frequency range, several dark blocks can be observed from the figure, indicating discontinuities in the data across different periods. Overall, these observations align with the corresponding components in the CEEMD decomposition discussed in the previous section. The captured features are consistent with the components shown in Fig. 3. Therefore, by combining Time2Vec and CEEMD, we can not only effectively filter out noise but also better capture both periodic and non-periodic patterns in stock data, thereby enhancing the predictive performance of the model.

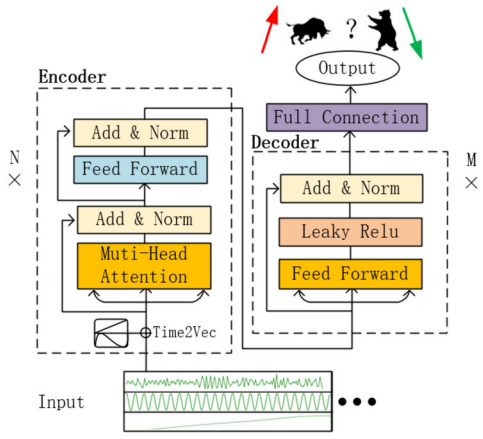

Figure 5 The architecture of Transformer in ETT

3.3.4 Transformer

Transformer consists of two main modules: the Encoder and the Decoder. The Encoder layer primarily includes Multi-Head Attention and Feed Forward Network. The former is used to capture relationships between features, while the latter facilitates further encoding learning. The Decoder module is responsible for decoding the data encoded by the Encoder. In ETT, multiple linear layers with leaky ReLU activation functions are used as the decoder of the model. Figure 5 illustrates the architecture of the Transformer used in ETT.

Mnih et al. [44] initially introduced attention mechanisms to extract information from an image or video by adaptively selecting a sequence of regions or locations. Vaswani et al. [10] proposed the self-attention mechanism, which computes the attention weights at each position during the encoding process of a sentence. The hidden vector representation of the entire sentence is then obtained by summing the weighted values. The specific formula is as follows:

$$\mathbf{Q} = \mathbf{Z} \times \mathbf{W}_Q \tag{12}$$

$$\mathbf{K} = \mathbf{Z} \times \mathbf{W}_K \tag{13}$$

$$\mathbf{V} = \mathbf{Z} \times \mathbf{W}_V \tag{14}$$

$$Attention\,(\mathbf{Q};\mathbf{K};\mathbf{V}) = softmax\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{model}}}\right)\mathbf{V} \tag{15}$$

$\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ represent the query, key, and value, respectively. $\mathbf{Q} \in \mathbb{R}^{T\times d_{model}}$, $\mathbf{K} \in \mathbb{R}^{T\times d_{model}}$, $\mathbf{V} \in \mathbb{R}^{T\times d_{model}}$.

The design of multi-head attention allows us to learn h sets of different linear projections to transform $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. These h sets of transformed queries, keys, and values are then attentively pooled in parallel. Finally, the outputs of these h attention pools are concatenated together and transformed by another learnable linear projection to obtain the final output, as shown in Fig. 6.

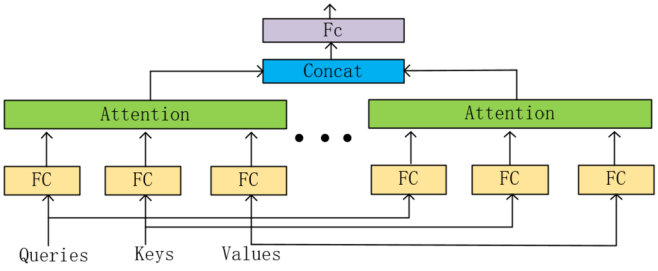

Figure 6 Architecture of the Multi-Head Attention based Transformer Model

Table 1 Basic information on ten selected features for stock analysis

Factor	Introduction	Category	Formula
Single day VPT	Daily Price-Volume Trends	Momentum	$VPT_t = VPT_{t-1} + (Closet_{n-k} - Closet_{t-1})/Closet_{t-1} * Volume_t$
EMA5	5-Day Exponential Moving Average	Technical	$EMA_N = \frac{2}{N+1}\sum_{k=0}^{n-1}(\frac{N-1}{N+1})^k Closet_{n-k}$
Kurtosis20	20-Day Kurtosis of Individual Stock Returns	Risk	$Kurtosis = \frac{\frac{1}{n}\sum_{i=1}^{n}(Closet_i - \bar{Closet_i})^4}{(\frac{1}{n}\sum_{i=1}^{n}(Closet_i - \bar{Closet_i})^2)^2}$
Momentum	Momentum Factor	Style	$MOMENTUM = Closet_i/Closet_{i-n} * 100\%$
BBIC	BBI Momentum	Momentum	$BBIC = (MA_3 + MA_6 + MA_{12} + MA_{24})/4$
PSY	Psychological Line Indicator	Emotion	$PSY = (N_{num\ of\ rising\ stocks}) * 100\%/N$
CCI10	10-day Chande Momentum Oscillator	Momentum	$CCI = (Closet - MA)/MD$
VOSC	Volume Oscillator	Emotion	$VOSC = (MA_{short} - MA_{long})/MA_{short}$
AR	Sentiment Indicator	Emotion	$AR = \sum_{i=1}^{n}(High_i - Low_i)/\sum_{i=1}^{n}(Open_i - Low_i)$
ROC12	12-day Rate of Change	Momentum	$ROC = (Closet_i - Closet_{i-n})/Closet_{i-n}$

4 Experiments

We selected two datasets from the A-share market, CSI 100 and Hushen 300, which cover stock trading data from January 1, 2019, to December 31, 2021. The data from January 1, 2019, to December 31, 2020, was treated as historical data and further divided into a training set and a validation set. Specifically, the training set comprises the first 70% of the data (from January 1, 2019, to June 30, 2020), while the remaining 30% (from July 1, 2020, to December 31, 2020) forms the validation set. The data from January 1, 2021, to December 31, 2021, was used as the backtesting set to evaluate and compare the predictive performance of ETT, Transformer, BiLSTM, LSTM, and CNN. Finally, we conducted a comprehensive backtesting analysis based on the predictions generated by each model.

4.1 Experimental setting

To validate the proposed algorithm, the CSI 100 and Hushen 300 datasets were obtained from Tushare. The daily data includes trading code, date, opening price, high price, low price, closing price, volume, and percent change. CSI 100 Index and Hushen 300 Index were used as baselines. The selected stock data covers the time range from January 1, 2019, to December 31, 2021. The features information for these stocks was obtained from Jukuan [45], and 99 features were selected based on their high correlation with the label and low inter-feature correlation. Table 1 presents examples of 10 selected features information. Subsequently, the trade data was right-joined with the Tushare dataset. Outliers

Table 2 Hyperparameter settings and optimal values for different models

	Range	CNNpred	SACLSTM	CNN-BiLSTM-AM	Deep Transformer	ETT
Batch size	[32, 256]	32	128	64	128	128
Epochs	[50, 500]	230	230	230	230	230
Learning rate	[0.0001, 0.1]	0.0001	0.0001	0.001	0.001	0.001
Dropout rate	[0, 0.5]	0.3	0.4	0.2	0.1	0.1
Encoder heads	[4, 16]	–	–	–	8	16
Hidden dim	[128, 512]	–	–	–	128	256

were handled for all data, and missing values were filled with the same value as the previous day, followed by normalization.

4.2 Hyper-parameters setting

To achieve optimal model performance, we employed a grid search strategy during the training process. The specific ranges for these hyperparameters are detailed in Table 2. By thoroughly examining various parameter combinations and independently training models using the SGD optimizer for each configuration, we evaluated the models' performance on the validation set. The optimal hyperparameters for each model are presented in Table 2.

Since CEEMD does not face the issue of selecting predefined basis functions as in wavelet transform [46], and Time2Vec also automatically captures the non-periodic representation of stock time series, there is no need for hyperparameter settings. Figure 7(a) and Fig. 7(b) illustrate the trend of the loss function for each validation set over the epochs. It can be observed that the model generally converges around 230 epochs on the validation set, after which the loss function rises at a very slow pace, further validating the parameter settings recommended by the grid search.

4.3 Training process and prediction results

Finally, the data is inputted into the adjusted Transformer for training and prediction. In the usage of deep learning models, it is common to evaluate the difference between actual values and predicted values using loss errors. Several commonly used metrics for predicting errors include mean absolute error (MAE), mean squared error (MSE), root mean square error (RMSE), and mean absolute percentage error (MAPE). The calculation formulas for these metrics are as follows:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{16}$$

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{17}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{18}$$

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}|\frac{y_i - \hat{y}_i}{y_i}| \times 100\% \tag{19}$$

where y_i is the true value, $\hat{y}_i$ is the predicted value, and n is the size of y_i.

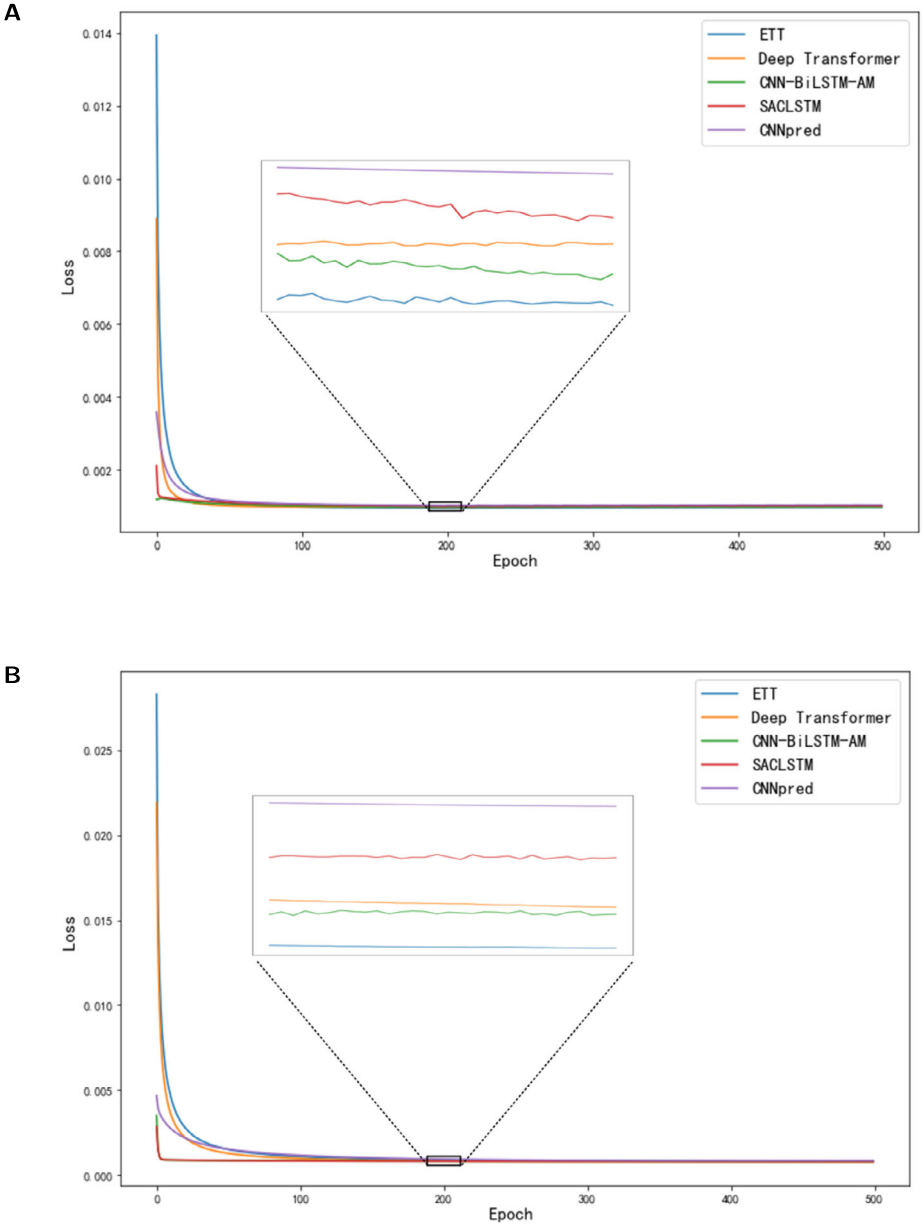

Figure 7 The loss functions of CSI 100 and Hushen 300 in validation set. (A) The loss functions of CSI 100 in the validation set. (B) The loss functions of Hushen 300 in the validation set

MAE, MSE, RMSE, and MAPE measure the predictive performance of the model from different perspectives. MAE and MSE quantify the absolute error of predictions, while MAPE assesses the relative error of the model. Smaller values of these metrics indicate better model performance. In this paper, the choice of prediction metric is the percent change, therefore MAE and MSE are selected as evaluation metrics. Table 3 presents the performance of various algorithms on different datasets.

From Table 3, it can be observed that the ETT model demonstrates better fitting performance than other compared deep learning models on the CSI 100 and Hushen 300 stock datasets. Whether from the perspective of MSE or MAE, ETT achieves the smallest pre-

Table 3 Prediction results based of different models on CSI 100 and Hushen 300 datasets. The better results on the validation set are highlighted in *bold* and the second best results are in underlined

		CSI 100		Hushen 300	
		MSE	MAE	MSE	MAE
CNNpred [49]	Training set	0.00080	0.02003	0.00070	0.01892
	Validation set	0.00103	0.02267	0.00082	0.01996
SACLSTM [50]	Training set	0.00078	0.01973	0.00069	0.01868
	Validation set	0.00101	0.02256	0.00079	0.01977
CNN-BiLSTM-AM [51]	Training set	0.00076	0.01965	0.00068	0.01850
	Validation set	0.00098	0.02246	0.00078	0.01966
Deep Transformer [34]	Training set	0.00077	0.01971	0.00068	0.01853
	Validation set	0.00099	0.02250	0.00078	0.01969
ETT	Training set	0.00075	0.01949	0.00067	0.01841
	Validation set	**0.00096**	**0.02237**	**0.00077**	**0.01955**
ETT without Time2Vec	Training set	0.00076	0.01966	0.00068	0.01851
	Validation set	0.00098	0.02248	0.00078	0.01969
ETT without CEEMD	Training set	0.00077	0.01970	0.00068	0.01852
	Validation set	0.00098	0.02249	0.00078	0.01967

diction errors among all the compared models on the datasets. The results of the ablation experiments indicate that adding the Time2Vec module reduces the MSE of predictions by 1.4% compared to the Transformer, adding the CEEMD module reduces the MSE by 1.7%, and the ETT model reduces the MSE by 2.7% compared to the Transformer. The ablation experiments demonstrate the effectiveness of the collaboration among the different modules. This is mainly because CEEMD can decompose the stock time series, which has large fluctuations, into smaller frequency components, while Time2Vec provides a time vector representation for the time series, capturing both periodic and non-periodic patterns while maintaining stability in response to time variations. The combination of these two modules with the Transformer leads to an improvement in the prediction effectiveness of stock sequences.

4.4 Backtesting

We have developed a stock backtesting strategy based on daily turnover to evaluate the predictive performance of the model. The strategy follows these steps:

(1) Clear the positions at the market open every day.

(2) Select the top n stocks with the highest predicted returns for the day and buy them. The buying weight w_i for the i-th stock is determined by its factor exposure to the predicted returns y_i.

$$w_i = e^{y_i} / \sum_{j=1}^{n} e^{y_j} \tag{20}$$

(3) If it is the last day, calculate the total value of stocks and cash. Otherwise, proceed to the next day.

n is set to 5, and the initial capital is set to 100,000. Transaction fees are not considered. The daily changes in total capital are observed. Figure 8 represents the performance of all algorithms on different datasets. Table 4 provides statistics on total returns, maximum drawdown, and Sharpe ratio for each algorithm. Total assets refer to the sum of stock and cash values after executing the strategy on the validation set. Maximum drawdown is the maximum decline in returns when the total assets reach their lowest point during the

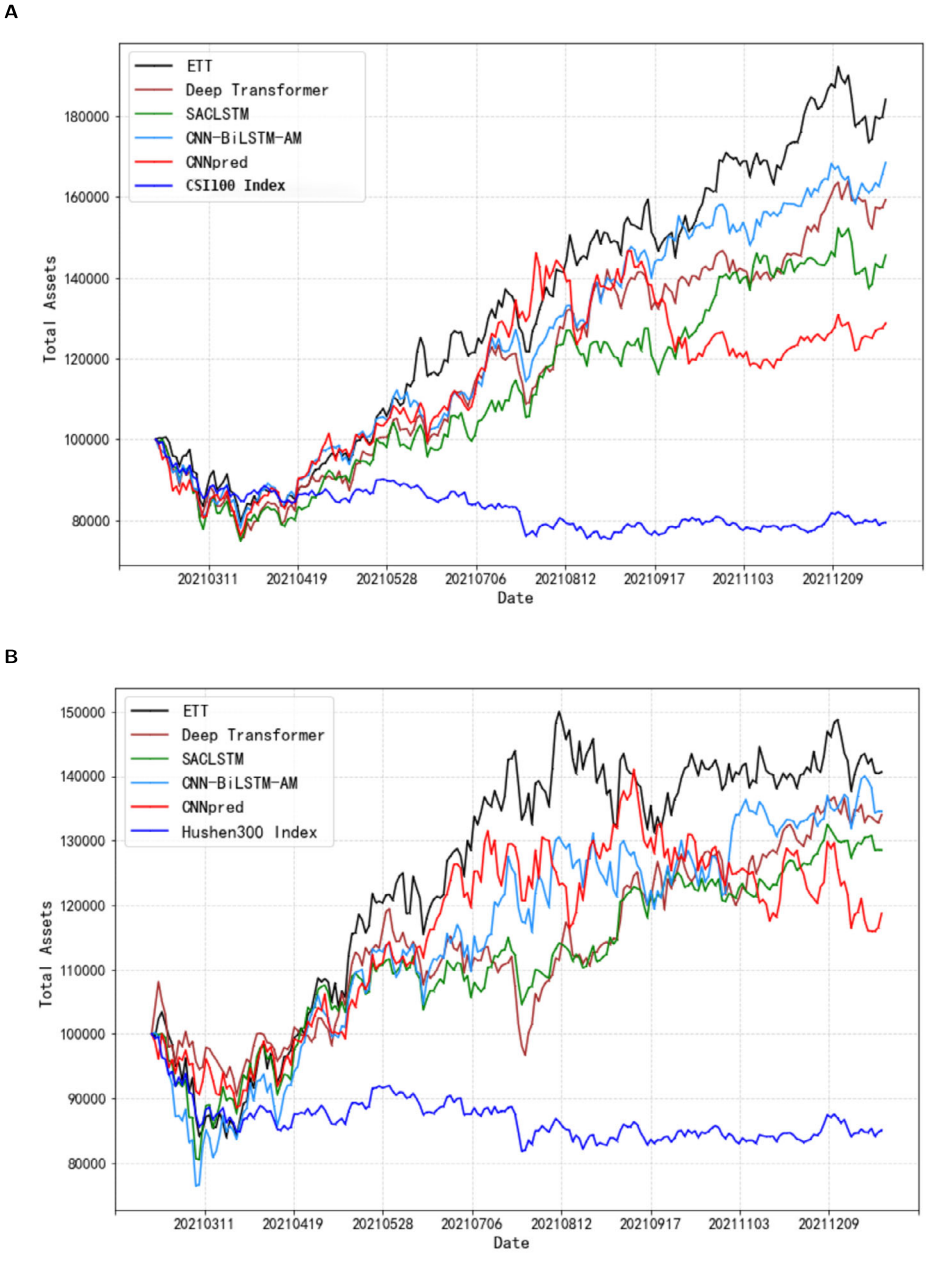

Figure 8 The predicted curves of the different models on CSI 100 and Hushen 300.(A) Backtesting on CSI 100. (B) Backtesting on Hushen 300

Table 4 The performance of the trading strategy on CSI 100 and Hushen 300. The better results are highlighted in *bold* and the second best results are in underlined

	CSI 100				Hushen 300			
	Total return	Max drawdown	Sharpe ratio	p-value (GW test)	Total return	Max drawdown	Sharpe ratio	p-value (GW test)
CNNpred	0.2876	**0.1978**	1.6176	0.30	0.1874	**0.1782**	1.2780	0.35
SACLSTM	0.4288	0.2613	1.9842	0.35	0.2855	0.1960	1.7879	0.41
CNN-BiLSTM-AM	0.6853	0.2188	2.8203	0.42	0.3457	0.2376	1.6077	0.48
Deep Transformer	0.5935	0.2484	2.3975	0.38	0.3402	0.1906	**1.9552**	0.46
ETT	**0.8416**	0.2079	**2.9120**	**0.50**	**0.4065**	0.1886	1.8021	**0.55**

backtesting period. It is an important risk indicator that describes the worst-case scenario for total assets. The Sharpe ratio [47] is a measure of risk-adjusted return for a portfolio, allowing for the comparison of risk and return across different portfolios. The formula for the Sharpe ratio is as follows:

$$Sharpe\ Ratio = \frac{E(R_p) - R_f}{\sigma_p} \tag{21}$$

Where $E(R_p)$ represents the expected annualized return of the portfolio, R_f is the annual risk-free rate, and σ_p is the standard deviation of the annualized return of the portfolio.

From Fig. 8 and Table 4, it can be observed that the ETT model achieves the highest cumulative returns for investors in the CSI 100 and Hushen 300 datasets. Although the CNNpred model sometimes accurately identifies stocks with high short-term returns, its poor robustness results in capturing stocks that decline, leading to the lowest cumulative returns. Additionally, the stock sequence array convolutional LSTM (SACLSTM) and CNN-BiLSTM-AM models exhibit relatively stable increases in total investment assets, but their risk-averse nature makes it challenging for them to capture stocks with exceptionally high returns during an upward market trend. The results of the Giacomini-White (GW) [48] test reveal that among the five models evaluated, the ETT model demonstrates superior predictive performance in both the CSI 100 and Hushen 300 datasets, with p-values of 0.50 and 0.55, respectively. This indicates that the ETT model's predictions are statistically more aligned with the actual values compared to the other models. The ETT model, on the other hand, is capable of capturing both periodic and non-periodic patterns, which aids in more accurate stock predictions. As a result, its increased returns are more stable, and it can more precisely capture stocks with short-term breakthrough potential during overall market upswings, thereby achieving higher cumulative returns.

5 Conclusion

This paper proposes and constructs the ETT model, which captures both periodic and non-periodic features in time series to assist in prediction. Through backtesting experiments conducted on the data from the CSI 100 and Hushen 300 datasets spanning from January 1, 2019, to December 31, 2021, the ETT model demonstrates significantly higher accuracy and cumulative returns compared to other benchmark deep learning models. This provides investors with more effective and reliable references. The effectiveness of each module of the model is further demonstrated through ablation studies conducted in this research.

However, there are some limitations to this study. It only considers the prediction of two Chinese stock datasets. The correlation between stock markets in many countries is high. Therefore, it would be meaningful to model and experiment with stock data from various countries worldwide. Additionally, exploring the application of the ETT model in other financial markets such as futures markets and bond markets would also be a valuable area for further exploration.

Abbreviations

CEEMD, Complete Ensemble Empirical Mode Decomposition; ETT, CEEMD-Time2Vec-Transformer; MSE, Mean Squared Error; GARCH, Generalized Autoregressive Conditional Heteroskedasticity; ARMA, Auto-Regressive Moving Average Model; SVM, Support Vector Machines; GBDT, Gradient Boosting Decision Trees; CNN, Convolutional Neural Networks; RNN, Recurrent Neural Networks; LSTM, Long Short-Term Memory Networks; BiLSTM, Bidirectional LSTM; IMF, Intrinsic

Mode Function; RF, Random Forest; PCA, Principal Component Analysis; MLP, Multilayer Perceptron; NSE, National Stock Exchange; NLP, Natural Language Processing; DSE, Dhaka Stock Exchange; NASDAQ, National Association of Securities Dealers Automated Quotations; EMD, Empirical Mode Decomposition; STNN, Stochastic Time Strength Neural Network; FCN, Fully Connected Neural Network; GRU, Gate Recurrent Unit; MAE, Mean Absolute Error; RMSE, root mean square error; MAPE, Mean Absolute Percentage error; SACLSTM, Stock Sequence Array Convolutional LSTM; AM, Attention Model.

Acknowledgements
Not applicable.

Author contributions
ZC proposed the central idea of the paper, drafted the manuscript, and led the overall design of the study. CJ conducted the data analysis, contributed to the interpretation of the results, and revised the manuscript. YS wrote the code and implemented the computational models. All authors read and approved the final manuscript.

Funding
This research was funded by the Major Humanities and Social Sciences Research Projects in Zhejiang higher education institutions, grant number 2023QN082 and the National Natural Science Foundation of China, grant number 61902349.

Availability of data and material
The datasets used or analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Competing interests
The authors declare that they have no competing interests.

Author details
[1]School of Economics, Zhejiang University of Technology, Hangzhou, China. [2]School of Computer Science, Zhejiang University of Technology, Hangzhou, China.

Received: 21 June 2023 Accepted: 19 December 2024 Published online: 03 January 2025

References

1. Kohara K, Fukuhara Y, Nakamura Y (1996) Selective presentation learning for neural network forecasting of stock markets. Neural Comput Appl 4:143–148
2. Moon K-S, Kim H (2019) Performance of deep learning in prediction of stock market volatility. Econ Comput Econ Cybern Stud Res 53:77–92
3. Bontempi G, Ben Taieb S, Le Borgne Y-A (2013) Machine learning strategies for time series forecasting. Business intelligence: second European summer school, eBISS 2012, Brussels, Belgium, July 15-21, 2012. Tutorial Lect 2:62–77
4. Polanco-Martínez JM (2019) Dynamic relationship analysis between nafta stock markets using nonlinear, nonparametric, non-stationary methods. Nonlinear Dyn 97(1):369–389
5. Wilhelmsson A (2006) Garch forecasting performance under different distribution assumptions. J Forecast 25:561–578
6. Lorenzato de Oliveira JF, Ludermir TB (2014) A distributed pso-arima-svr hybrid system for time series forecasting. In: 2014 IEEE international conference on Systems, Man, and Cybernetics (SMC), pp 3867–3872
7. Xu X, Chen Y (2001) Empirical study on nonlinearity in China stock market. Quantit Tech Econ 18:110–113
8. Kim K-j (2003) Financial time series forecasting using support vector machines. Neurocomputing 55(1):307–319
9. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521(7553):436–444
10. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Adv Neural Inf Process Syst 30
11. Zhou F, Zhang Q, Zhu Y, Li T (2023) T2v_tf: an adaptive timing encoding mechanism based transformer with multi-source heterogeneous information fusion for portfolio management: a case of the Chinese a50 stocks. Expert Syst Appl 213:119020
12. Kazemi SM, Goel R, Eghbali S, Ramanan J, Sahota J, Thakur S, Wu S, Smyth C, Poupart P, Brubaker M (2019) Time2vec: Learning a vector representation of time. arXiv preprint. arXiv:1907.05321
13. Wang K, Qi X, Liu H, Song J (2018) Deep belief network based k-means cluster approach for short-term wind power forecasting. Energy 165:840–852
14. Wang J, Cui Q, Sun X, He M (2022) Asian stock markets closing index forecast based on secondary decomposition, multi-factor analysis and attention-based lstm model. Eng Appl Artif Intell 113:104908
15. Masset P (2008) Analysis of financial time-series using Fourier and wavelet methods. Available at SSRN 1289420
16. Teo KK, Wang L, Lin Z (2001) Wavelet packet multi-layer perceptron for chaotic time series prediction: effects of weight initialization. In: Computational science - ICCS 2001. Springer, Berlin, pp 310–317
17. Huang NE, Shen Z, Long SR, Wu MC, Shih HH, Zheng Q, Yen N-C, Tung CC, Liu HH (1998) The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis. Proc R Soc Lond, Ser A, Math Phys Eng Sci 454(1971):903–995
18. Siami-Namini S, Tavakoli N, Namin AS (2019) The performance of lstm and bilstm in forecasting time series. In: 2019 IEEE international conference on big data (big data). IEEE, pp 3285–3292
19. Chen L, Chi Y, Guan Y, Fan J (2019) A hybrid attention-based emd-lstm model for financial time series prediction. In: 2019 2nd International Conference on Artificial Intelligence and Big Data (ICAIBD). IEEE, pp 113–118

20. Chen L, Chi Y, Guan Y, Fan J (2019) A hybrid attention-based emd-lstm model for financial time series prediction. In: 2019 2nd International Conference on Artificial Intelligence and Big Data (ICAIBD), pp 113–118
21. Yao Y, Zhang Z-y, Zhao Y (2023) Stock index forecasting based on multivariate empirical mode decomposition and temporal convolutional networks. Appl Soft Comput 142:110356
22. Long W, Lu Z, Cui L (2019) Deep learning-based feature engineering for stock price movement prediction. Knowl-Based Syst 164:163–173
23. Kaczmarek T, Perez K (2022) Building portfolios based on machine learning predictions. Econ Res 35(1):19–37
24. Yu H, Chen R, Zhang G (2014) A svm stock selection model within pca. Proc Comput Sci 31:406–412
25. Sunny MAI, Maswood MMS, Alharbi AG (2020) Deep learning-based stock price prediction using lstm and bi-directional lstm model. In: 2020 2nd Novel Intelligent and Leading Emerging Sciences conference (NILES). IEEE, pp 87–92
26. Hu Z, Zhao Y, Khushi M (2021) A survey of forex and stock price prediction using deep learning. Appl Syst Innov 4(1):9
27. Takeuchi L, Lee Y-YA (2013) Applying deep learning to enhance momentum trading strategies in stocks. Stanford University, Stanford
28. Sirignano J, Cont R (2019) Universal features of price formation in financial markets: perspectives from deep learning. Quant Finance 19(9):1449–1459
29. Kaushik M, Giri A (2020) Forecasting foreign exchange rate: a multivariate comparative analysis between traditional econometric, contemporary machine learning & deep learning techniques. arXiv preprint. arXiv:2002.10247
30. Selvin S, Vinayakumar R, Gopalakrishnan E, Menon VK, Soman K (2017) Stock price prediction using lstm, rnn and cnn-sliding window model. In: 2017 International Conference on Advances in Computing, Communications and Informatics (icacci). IEEE, pp 1643–1647
31. Li S, Jin X, Xuan Y, Zhou X, Chen W, Wang Y-X, Yan X (2019) Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. Adv Neural Inf Process Syst 32
32. Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W (2021) Informer: beyond efficient transformer for long sequence time-series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol 35, pp 11106–11115
33. Muhammad T, Aftab AB, Ahsan M, Muhu MM, Ibrahim M, Khan SI, Alam MS, et al (2022) Transformer-based deep learning model for stock price prediction: a case study on Bangladesh stock market. arXiv preprint. arXiv:2208.08300
34. Wang C, Chen Y, Zhang S, Zhang Q (2022) Stock market index prediction using deep transformer model. Expert Syst Appl 208:118128
35. Ding Q, Wu S, Sun H, Guo J, Guo J (2020) Hierarchical multi-scale Gaussian transformer for stock movement prediction. In: IJCAI, pp 4640–4646
36. Malibari N, Katib I, Mehmood R (2021) Predicting stock closing prices in emerging markets with transformer neural networks: the saudi stock exchange case. Int J Adv Comput Sci Appl 12(12)
37. Rezaei H, Faaljou H, Mansourfar G (2021) Stock price prediction using deep learning and frequency decomposition. Expert Syst Appl 169:114332
38. Xian L, He K, Wang C, Lai KK (2020) Factor analysis of financial time series using eemd-ica based approach. Sustain Futures 2:100003
39. Wang J, Wang J (2017) Forecasting stochastic neural network based on financial empirical mode decomposition. Neural Netw 90:8–20
40. Dang H, Mei B (2022) Stock movement prediction using price factor and deep learning. Int J Comput Inf Eng 16(3):73–76
41. Torres ME, Colominas MA, Schlotthauer G, Flandrin P (2011) A complete ensemble empirical mode decomposition with adaptive noise. In: 2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, pp 4144–4147
42. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. arXiv preprint. arXiv:1301.3781
43. Tallec C, Ollivier Y (2018) Can recurrent neural networks warp time? arXiv preprint. arXiv:1804.11188
44. Mnih V, Heess N, Graves A, et al (2014) Recurrent models of visual attention. Adv Neural Inf Process Syst 27
45. JoinQuant (2014) JoinQuant Quantitative Investment Platform. Accessed: 2024-09-21. https://www.joinquant.com
46. Sović A, Seršić D (2012) Signal decomposition methods for reducing drawbacks of the dwt. Eng Rev 32(2):70–77
47. Sharpe WF (1966) Mutual fund performance. J Bus 39(1):119–138
48. Giacomini R, White H (2006) Tests of conditional predictive ability. Econometrica 74(6):1545–1578
49. Hoseinzade E, Haratizadeh S (2019) Cnnpred: cnn-based stock market prediction using a diverse set of variables. Expert Syst Appl 129:273–285
50. Wu JM-T, Li Z, Herencsar N, Vo B, Lin JC-W (2021) A graph-based cnn-lstm stock price prediction algorithm with leading indicators. Multimed Syst:1–20
51. Lu W, Li J, Wang J, Qin L (2021) A cnn-bilstm-am method for stock price prediction. Neural Comput Appl 33:4741–4753

Publisher's Note